

Q1: Distributed Rate Limiter Service

Coding Interview: Distributed Rate Limiter REST API Service

Mandatory Environment Rules

To ensure your work is saved and evaluated correctly, you must adhere to the following operational constraints:

- Stay in the Workspace: All development must occur inside the /workspaces directory within your IDE.
- No Host Saving: Do not save code or assets to the host machine's Desktop or Downloads folder. These areas are not bridged to the container; data saved there will be permanently lost when the session ends.
- Final Submission: You must push your code and architecture diagram to a personal GitHub repository before the timer expires.
- Session Cleanup: Sign out of all personal accounts (GitHub, Cursor, Browser) before handing back the workstation.

Project Objective

Summary: Build a production-ready, horizontally scalable rate limiting service that enforces configurable per-tenant request quotas across multiple instances, exposes an evaluation API, and surfaces observable quota signals.

Functional Expectations

At a minimum, your service should demonstrate:

- Quota Enforcement: Accept an identifier (e.g. tenant ID, API key) and a resource namespace. Return an allow/deny decision with remaining quota and reset time. Support at least two algorithm choices (e.g. token bucket, sliding window, fixed window).
- Configuration API: Allow rate limit rules to be created, updated, and deleted at runtime without restarting the service. Rules should support burst and sustained rate distinctions.
- Multi-instance Correctness: The service must produce correct decisions when two or more instances share state — not just when running as a single process.
- Observable Quota State: Provide an endpoint returning current quota consumption, remaining capacity, and reset timestamps for a given identifier.

Engineering Expectations

Your solution should reflect what you believe constitutes a production-ready service. We require:

- Architecture Flow Diagram: Include a diagram in your repository mapping the request lifecycle across instances, the shared state layer, and the evaluation path.
- Correctness Under Contention: Demonstrate — through tests or reasoning — how your implementation handles concurrent evaluations for the same key. Where are the race windows, and how are they bounded?

- Testing: Unit tests for algorithm logic and integration tests exercising the HTTP layer. At least one test must exercise concurrent access.
- CI/CD: A functional GitHub Actions pipeline that runs your tests on push.
- Documentation: README covering setup, algorithm selection rationale, and known limitations of your design.

Extensions & Next Steps

If time permits, you are encouraged to expand on your solution:

- Deployment: Deploy to DigitalOcean with at least two replicas behind a load balancer and demonstrate cross-instance correctness.
- Adaptive Limits: Add a feedback signal (e.g. downstream error rate) that temporarily tightens or relaxes limits for a tenant.
- Admin Dashboard: A simple UI showing real-time quota utilization per tenant.

Good luck. We look forward to reviewing your solution.